Definitions

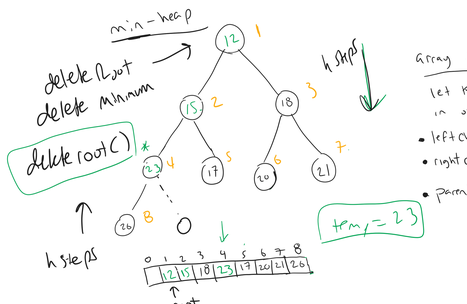
1) A binary tree of height h, is full if there are 2^h nodes at depth h.

2) A binary tree is complete if the tree of height h-1 is full, and the bottom-most level is filled from left to right.

3) min heap order property: if every node is smaller than or equal to its two children.

for max heap every node is larger than or equal to its children.

4) A Heap is a complete binary tree with the heap-order property.



Array traversal rules

let K be any node in our heap. n is the number of nodes.

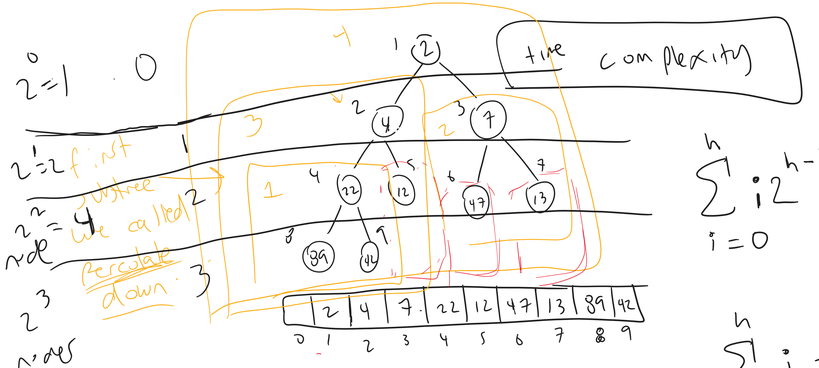
- leftchild(k) = 2k ; $k \leq \frac{n}{2}$
- rightchild(k) = 2k+1 ; $k \leq \frac{n}{2}$
- parent(k) = $\lfloor \frac{k}{2} \rfloor$ $k > 1$

$h = \log(n)$

functions for heap datastructure

1) insert → insert any element maintain heap order property
2) deleteRoot → heaps can only delete root. maintain heap property after delete.
* 3) heapify → convert an array into a heap.

runtime: $O(\log n)$ for insert and deleteroot



$$\sum_{i=0}^{h} i\,2^{h-i} = \#\text{ of subtrees}$$

→ # of trees we called percolate down

$$\sum_{i=0}^{h} i\,\frac{2^h}{2^i} = 2^h \sum_{i=0}^{h} \frac{i}{2^i}$$

geometric series → converges to two

$$= 2^h (2)$$

$$= 2^{\log n} (2)$$

$$= n \cdot (2) \in O(n)$$

heapify

$2^{\log n} = n$